

Production Readiness Gaps — Boiler IoT Simulation

Audit of current implementation vs. what a production-grade industrial IoT + Agentic RAG platform requires. Each gap lists **status today**, **risk if shipped as-is**, and **what to build**.

1. Security & Access Control

#	Gap	Status today	Risk	What to build
1.1	MQTT broker open (anonymous)	EMQX_ALLOW_ANONYMOUS=true	Anyone on network can publish fake sensor values or subscribe to all topics	EMQX auth: per-device certs (mTLS) or username/password + ACL per topic prefix
1.2	Hardcoded secrets in <code>docker-compose.yml</code>	InfluxDB token <code>my-super-secret-token-123</code> , Grafana <code>admin123</code>	Token leak via git history → full DB read/write	Move to <code>.env</code> + secrets manager (Vault / AWS Secrets Manager / GCP Secret Manager); rotate tokens; never commit
1.3	No TLS anywhere	MQTT plain 1883, Influx HTTP, Grafana HTTP	Credential + telemetry sniffable on LAN	MQTT 8883 TLS, Influx HTTPS, Grafana behind reverse proxy with TLS (nginx/traefik + Let's Encrypt)
1.4	No API auth on assistant/chat API	<code>api/</code> endpoints open	Anyone can hit LLM endpoint → cost abuse, prompt injection at scale	JWT / OAuth2 (Auth0, Keycloak), per-user rate limits, API keys for service-to-service
1.5	No RBAC	Single admin user everywhere	Operator vs. engineer vs. read-only viewer all collapsed	Roles: viewer, operator, engineer, admin. Enforce in API + Grafana orgs
1.6	No audit log	No record of who acknowledged which fault, who queried the assistant	Compliance failure (IBR + ISO 27001 require traceability)	Append-only audit log (DB table or SIEM) for: logins, fault acks, config changes, LLM queries
1.7	LLM prompt	User input goes	Malicious user can	Input sanitisation,

#	Gap	Status today	Risk	What to build
	injection unguarded	straight into Gemini	exfiltrate system prompt or pivot tool calls	output filtering, allow-list tool args, separate channels for system vs. user content

2. Reliability & High Availability

#	Gap	Status today	Risk	What to build
2.1	Single MQTT broker	One EMQX container, no clustering	Broker death = full data loss until restart	EMQX cluster (3 nodes), shared subscriptions for load-balanced consumers
2.2	Single InfluxDB instance	One container, no replication	Disk failure = all telemetry gone	InfluxDB Enterprise / Cloud, or replicate to TimescaleDB cluster; periodic snapshots to S3
2.3	Consumers single-instance, no restart policy beyond compose	<code>fault_detect</code> or <code>.py</code> , <code>influx_consumer.py</code> run once	Process crash on bad payload (we saw <code>datetime.Now</code>) silently stops fault detection	Run as systemd / Kubernetes Deployments with <code>replicas≥2</code> , healthchecks, auto-restart
2.4	No dead-letter queue	Bad MQTT payload → exception → message dropped	Lost faults, no replay	DLQ topic <code>system/dlq/*</code> ; consumer publishes unparseable messages there with reason
2.5	No circuit breaker on Influx writes	<code>write_api.write(...)</code> blocks consumer if Influx down	Back-pressure stalls MQTT consumer, broker queue fills	Async batch writes, retry with exponential backoff, circuit breaker (pybreaker / tenacity)
2.6	No graceful shutdown	<code>loop_forever()</code> with no SIGTERM	Container kill mid-write corrupts batch	Signal handlers; drain in-flight writes before exit

#	Gap	Status today	Risk	What to build
		handler		

3. Observability

#	Gap	Status today	Risk	What to build
3.1	Only <code>print()</code> logs	Console only, lost on container restart	Cannot debug production incidents	Structured logging (loguru / structlog) → JSON → Loki / ELK / Cloud Logging
3.2	No metrics	No Prometheus, no /metrics endpoint	Cannot see consumer lag, write throughput, LLM latency, tool error rate	<code>prometheus_client</code> : counters for <code>messages_in/faults_raised/llm_calls/tool_errors</code> ; histograms for <code>write_latency/llm_latency</code>
3.3	No distributed tracing	LLM → tool → Influx chain opaque	"Why was that answer slow?" unanswerable	OpenTelemetry SDK in <code>orchestrator.py</code> + tools; export to Tempo / Jaeger / Cloud Trace
3.4	No alerting on the monitoring stack itself	Grafana dashboards exist for sensor data, none for system health	Silent failure: fault detector dies, no one notices	Alertmanager rules: consumer lag > N, Influx down, MQTT broker disconnects, LLM error rate spike
3.5	No SLO definition	No targets for ingestion latency, fault-detection delay, LLM p95	Cannot prove production quality to stakeholders	Define SLOs: e.g. fault detected within 2s of breach in 99.9% of cases; LLM p95 < 8s

4. Data Pipeline & Storage

#	Gap	Status today	Risk	What to build
4.1	No retention policy	Influx bucket grows unbounded	Disk fills, ingestion stops	Downsampling tasks: raw 7 days → 1-min aggregates 90 days → 1-hour aggregates 2 years
4.2	No backup	No influx backup cron	Disaster = total loss	Daily snapshot to S3/GCS, encrypted, retention 30 days; test restore quarterly
4.3	No schema versioning	Sensor names hardcoded in BOILER_RULES and tool schemas	Renaming a sensor breaks fault detector, agent tool schema, dashboards independently	Single source of truth (YAML/JSON) loaded by simulator + detector + tool schema + Grafana provisioning
4.4	No data quality checks	Negative steam flow, NaN, frozen sensor not detected	Stale/garbage data silently feeds LLM	Validation layer: range checks, staleness detector (no update > N seconds → STALE status), spike detector
4.5	No event sourcing for faults	Faults written as InfluxDB points only	Cannot reconstruct fault lifecycle (raised → acked → resolved → recurrence)	Separate fault state DB (Postgres) with state machine: OPEN/ACKED/RESOLVED/SUPPRESSED

5. Agentic RAG / LLM Layer

#	Gap	Status today	Risk	What to build
5.1	No knowledge base tool	search_knowledge_base is TODO in tool_schemas.py	Agent cannot cite IBR / SOPs / equipment manuals	Vector DB (pgvector / Pinecone / Weaviate) of IBR docs, OEM

#	Gap	Status today	Risk	What to build
				manuals, incident reports; embed with text-embedding model; hybrid search (BM25 + vector)
5.2	No conversation memory	Each <code>run()</code> is stateless	Cannot do follow-up questions ("and what about chimney?")	Session store (Redis): last N turns per user; pass into orchestrator
5.3	No tool result caching	Same <code>fetch_realtime_sensors</code> query per question hits Influx every time	Cost + latency at scale	Short-TTL cache (Redis 2-5s for realtime, 60s for fault history)
5.4	No LLM response evaluation in production	<code>evaluation/</code> folder exists but offline only	Quality regressions ship silently	Online eval: sample 1% of responses → LLM-as-judge (groundedness, helpfulness) → metrics; tie to alerts
5.5	No guardrails on tool outputs	Tool result string interpolated raw	Influx error message could contain user-controlled text → injection	Schema-validate tool outputs; truncate; strip control tokens
5.6	No cost tracking	Vertex AI calls untracked	Surprise bill, no per-user quota	Token counter per request → Prometheus + per-user daily budget
5.7	No fallback when LLM down	Vertex error → API 500 to user	Outage = total feature loss	Fallback path: deterministic templated answer from tool results when LLM unavailable
5.8	Fine-tuned endpoint single region	<code>FINE_TUNED_MODEL_ENDPOINT</code> one region	Regional outage = down	Multi-region endpoint with failover; warm standby

6. Edge / Device Layer

#	Gap	Status today	Risk	What to build
6.1	Simulated data only	<code>boiler_simulator.py</code> , <code>chimney_simulator.py</code> synthetic	Not production until real PLC/SCADA integration	OPC-UA / Modbus TCP gateway → MQTT bridge; map real tag names to current sensor names
6.2	No device identity / provisioning	Hardcoded <code>BOILER_001</code> , <code>CHIMNEY_001</code>	Cannot scale to N plants	Device registry, per-device certs, MQTT client ID = device ID, ACL per device
6.3	No edge buffering	Simulator publishes direct; if broker down → drop	Real plant cannot lose telemetry during network blip	Edge agent with local SQLite/file buffer; forward when broker reachable
6.4	No firmware / config OTA	N/A	Cannot push threshold updates to fleet	Config topic per device, version-stamped; device acks

7. CI/CD & DevEx

#	Gap	Status today	Risk	What to build
7.1	No automated tests in CI	No <code>pytest</code> runs, no GitHub Actions visible	Bugs like <code>datetime.Now</code> reach runtime	Unit tests for rules, integration tests with testcontainers (MQTT + Influx), CI on PR
7.2	No type checking	No mypy / pyright	Refactor breakage	<code>mypy --strict</code> on <code>consumers/</code> , <code>assistant/</code>
7.3	No linting / formatting gate	Mixed style	Bus factor, review noise	<code>ruff</code> + <code>black</code> + pre-commit hooks
7.4	No container scanning	Base images unscanned	CVE in <code>emqx:latest</code> , <code>influxdb:2.7</code>	Trivy / Snyk in CI; pin image digests, not <code>:latest</code>
7.5	<code>latest</code> tags	<code>emqx:latest</code> ,	Non-reproducible	Pin versions:

#	Gap	Status today	Risk	What to build
	everywhere	grafana:latest	builds, surprise breakages	emqx:5.4.1, grafana:10.4.2
7.6	No staging environment	Single docker-compose	Testing changes against prod data	dev / staging / prod with separate state
7.7	Dependency hygiene	requirements.txt + pyproject.toml + uv.lock coexist	Drift between envs	Single source: pyproject.toml + uv.lock; remove requirements.txt or autogenerate

8. Operational Tooling

#	Gap	Status today	Risk	What to build
8.1	No fault acknowledgement workflow	Faults publish to system/faults → nothing	Operators cannot ack, suppress, or annotate	Web UI: ack button writes to fault state DB; suppression with TTL; runbook link per fault code
8.2	No notification channel	Console print only	On-call engineer not paged	PagerDuty / Opsgenie / Slack webhook for CRITICAL; email for WARNING
8.3	No runbooks linked	Fault code → no SOP	MTTR depends on tribal knowledge	Markdown runbook per fault code, linked from Grafana alert + assistant answer
8.4	No maintenance mode	Cannot mute a sensor during planned work	Alarm storms during maintenance	Suppression API: mute sensor X for N hours, audit-logged
8.5	Streamlit demo UI only	streamlit_app.py good for demo, not for control room	Not multi-user, no auth, no responsive design	Production frontend: Next.js / React + auth + role-based views

9. Compliance (IBR / ISO)

#	Gap	Status today	Risk	What to build
9.1	No tamper-evident logs	Influx is mutable with token	Auditor rejects	Write-once storage (S3 Object Lock) for fault events + audit log
9.2	No data residency control	Vertex region in config but data path unverified	India IBR may require local processing	Document + enforce: where does telemetry / LLM payload travel; offer region-locked deployment
9.3	No retention policy doc	Implicit	Audit failure	Written policy: telemetry 2 years, faults 7 years, audit log 7 years
9.4	No PII / sensitive-data review	LLM logs full prompts	Operator names, plant names in logs	PII scrubber on log path; configurable redaction

10. Scalability

#	Gap	Status today	Risk	What to build
10.1	Single-plant assumption	One BOILER_001 + CHIMNEY_001	Cannot onboard plant #2 without rewriting	Multi-tenant model: <code>tenant_id</code> tag on every topic, point, fault; assistant scoped by tenant
10.2	Consumer not horizontally scalable	Each consumer is one process subscribing to all topics	Bottleneck at scale	EMQX shared subscriptions (<code>\$share/group/topic</code>); N consumer replicas
10.3	Fault rules hardcoded in code	<code>BOILER_RULES</code> dict in <code>fault_detect</code> or <code>.py</code>	Rule change = redeploy	Rules in DB or hot-reloaded YAML; UI for engineers to edit
10.4	No load test	Unknown ceiling	Surprise failure under real load	k6 / Locust scenario: N devices $\times$ M

#	Gap	Status today	Risk	What to build
				sensors × 1Hz; measure end-to-end latency

Suggested Build Order (high impact, low effort first)

1. **Secrets + TLS + MQTT auth** (1.1–1.3) — non-negotiable before any real device
2. **Structured logging + Prometheus metrics** (3.1, 3.2) — every later fix depends on visibility
3. **CI with tests, type check, lint** (7.1–7.3) — stop bugs like `datetime.Now`
4. **Schema source-of-truth + sensor staleness detection** (4.3, 4.4) — fix silent data corruption
5. **Knowledge base tool + conversation memory** (5.1, 5.2) — biggest assistant UX jump
6. **Fault state machine + ack workflow + notifications** (4.5, 8.1, 8.2) — turn alerts into incident response
7. **HA: cluster broker, replica consumers, restart policies** (2.1–2.5)
8. **Backups + retention + downsampling** (4.1, 4.2)
9. **Multi-tenant + horizontal scale** (10.1, 10.2)
10. **Compliance + audit logs** (1.6, 9.x)

Target Architecture (one-liner per layer)

- **Edge:** OPC-UA → edge agent (buffer + mTLS) → MQTT
- **Ingest:** EMQX cluster (mTLS, ACL, shared subs) → consumers (K8s Deployments, replicas, DLQ)
- **Storage:** InfluxDB cluster (hot 7d) + S3 Parquet (cold, Object Lock) + Postgres (fault state, audit)
- **Detection:** rule engine with hot-reload YAML + DB-backed fault state machine
- **Assistant:** Vertex fine-tuned Gemini + pgvector KB + Redis session/cache + OpenTelemetry + guardrails + cost meter
- **API:** FastAPI + JWT + rate limit + Prometheus + OpenAPI
- **Frontend:** Next.js + role-based views + ack workflow + runbook links
- **Ops:** Loki + Prometheus + Grafana + Alertmanager → PagerDuty
- **CI/CD:** GitHub Actions → tests + mypy + ruff + Trivy → image registry → ArgoCD →

K8s